Build Your Personal Business Dashboard

With Claude Code: From Zero to a Working App

Google Drive | Google Sheets | OpenAI | Gmail | Calendar

Working Series • Hands-On • No Coding Experience Required

THE PROBLEM

Your Business Data Is Everywhere

You're running businesses. Your data lives in scattered spreadsheets, inboxes, and folders. Sound familiar?

Leads

In a Google Sheet somewhere. Maybe two. Different formats. Some are outdated. You know there are duplicates but who has time?

Invoices

In Drive. In email. In your Downloads folder. Who's paid? Who hasn't? Follow-ups fall through the cracks every month.

Projects

Tracked in your head. Or a spreadsheet that's 3 weeks behind reality. No clear link between projects, leads, and invoices.

> *"I don't need more tools. I need the tools I already use to talk to each other."*

THE SOLUTION

One Dashboard. All Your Data. Actionable.

By the end of this series, you'll have a working dashboard that:

Auto-Organizes

Drop a spreadsheet into Google Drive. The pipeline normalizes it into your master database, no manual cleanup.

Cross-References

Upload an invoice for "Acme Corp" and the system finds the matching lead and marks them as "converted."

Emails Directly

Click a button on a lead to send them an email, right from the dashboard. No switching apps.

Sets Reminders

One click adds an invoice due date to your Google Calendar with a 24-hour reminder.

> *And you're going to build it using Claude Code, by describing what you want in plain English.*

THE TOOL

What Is Claude Code?

A terminal tool that runs on your machine. You type plain English. It reads and writes your actual project files.

```
$ claude

You: Build a /leads page that reads
     from the Sheets service and shows
     a table with name, email, company,
     and a stage badge.

Claude Code: I'll create the template,
  add the route, and wire it to the
  sheets service.

  ✓ Created templates/leads.html
  ✓ Updated routes/dashboard_routes.py
  ✓ Updated controllers/dashboard_controller.py
```

It sees your files

Claude Code reads your project structure, understands what exists, and edits the right files in the right places.

You describe, it builds

No syntax to learn. No copy-pasting from Stack Overflow. Say what you want and watch it happen.

It runs on your machine

Your files stay local. Nothing is uploaded to a website. It works in your terminal, on your project.

It's not a chatbot. *It sees your files. It knows your project. When you say "add a button to the leads page," it knows exactly which file to edit.*

SESSION 1

Getting Started with Claude Code

Setting up your editor, installing Claude Code, and logging in

This is where we make sure everyone is up and running

SESSION 1, STEP 1

What is a Code Editor?

A code editor is the app where you view and edit your project files. Think of it like Microsoft Word, but for code.

Visual Studio Code (VS Code)

A free code editor made by Microsoft. The most popular editor in the world. Over 70% of developers use it.

Download: `code.visualstudio.com`

Cursor

An AI-focused code editor built on top of VS Code. Looks and works the same way, with extra AI features built in.

Download: `cursor.com`

Why these two? *They're free, beginner-friendly, and widely supported. There are other editors out there (Sublime Text, WebStorm, Vim) and Claude Code works in all of them. We chose these because they're the easiest starting point.*

SESSION 1, STEP 2

Open Your Editor

Find the app on your computer and open it. Here's what you'll see:

On Mac

1. Open **Finder**
2. Go to **Applications**
3. Double-click **Visual Studio Code** (or **Cursor**)

On Windows

1. Click the **Start menu** (bottom-left of screen)
2. Search for **Visual Studio Code** (or **Cursor**)
3. Click to open it

> ***You should see a welcome screen.*** *It might look overwhelming with lots of panels and icons. That's okay. We only need two things: the file explorer on the left, and the terminal at the bottom.*

SESSION 1, STEP 3

Open a Project Folder

A "project" is just a folder on your computer. Claude Code needs to know which folder to work in.

1. **If you already have a project folder:**
   In your editor, go to **File** in the top menu bar, then click **Open Folder**. Navigate to your project folder and select it.

2. **If you're starting from scratch:**
   First, create a new folder. Open Finder (Mac) or File Explorer (Windows). Go to your Desktop or Documents. Right-click, choose **New Folder**, and name it something like `my-dashboard`. Then go back to your editor and open that folder with **File → Open Folder**.

> ***Why does this matter?*** *When Claude Code starts, it looks at all the files in your open folder. That's how it knows what your project is. If you open the wrong folder, or no folder at all, Claude won't know what you're working on.*

SESSION 1, STEP 4

Open the Terminal

The terminal is a text-based window where you type commands. Your editor has one built in.

1. In the top menu bar of your editor, click **View**
2. Click **Terminal**
3. A panel appears at the bottom of your editor. This is your terminal. You'll see a blinking cursor waiting for you to type.
4. Type `claude` and press **Enter**

Pro tip: If you already have Claude Code installed and you see the Claude logo appear, you're good to go. Skip the next slide and jump straight to logging in.

If you see "command not found" or nothing happens, that's okay. It just means Claude Code isn't installed yet. The next slide walks you through it step by step.

SESSION 1, STEP 5

First: Install Node.js

Claude Code requires Node.js to run. Node.js is a free tool that lets your computer run JavaScript programs. You only need to install it once.

Check if you already have it:

In your terminal, type this and press Enter:

```
node --version
```

See a version number (like v20.11.0)? You're good. Skip to the next slide.

See "command not found"? You need to install Node.js. Follow the steps on the right.

Install Node.js:

1. Open your web browser
2. Go to `nodejs.org`
3. Click the big green **LTS** download button (LTS means "stable version")
4. Run the installer. Click Next through all the steps. Accept the defaults.
5. **Close and reopen your terminal**, then try `node --version` again

SESSION 1, STEP 6

Install Claude Code

Now that Node.js is installed, one command gets you Claude Code.

1. In your terminal, type this and press Enter:

```
npm install -g @anthropic-ai/claude-code
```

This downloads and installs Claude Code globally on your computer. It may take a minute.

2. Verify it worked:

```
claude --version
```

You should see a version number like `2.1.92`

Still seeing "claude: command not found"? This is the #1 issue. It means your system can't find where Claude Code was installed.

Mac fix: Run: `echo 'export PATH="$HOME/.npm-global/bin:$PATH"' >> ~/.zshrc && source ~/.zshrc`

Windows fix: Close PowerShell completely and reopen it. If still broken, restart your computer.

Stuck? Open `claude.ai` in your browser and start a new chat. Paste this: *"I'm trying to install Claude Code on my [Mac/Windows]. I ran the install command but I'm getting [your error]. Help me fix it step by step."* Claude can debug your specific setup.

SETUP

Log In to Claude Code

You do NOT need an API key. Your Anthropic subscription (Max plan) includes Claude Code. Let's log in right now.

Do this now, step by step:

1. Open **VS Code** (or Cursor) → open the terminal
2. Type `claude` and press Enter. You'll see this:

```
$ claude
▐▛▜▌  Claude Code v2.1.92
▝▜▛▘  Opus 4.6 (1M context) · Claude Max
      ~/Documents

❯ /login

  /login            Sign in with your Anthropic account
  /plugin           Manage Claude Code plugins
  /reload-plugins   Activate pending plugin changes
```

3. Type `/login`. You'll see the login options:

```
▐▛▜▌  Claude Code v2.1.92
▝▜▛▘  Opus 4.6 (1M context) · Claude Max

 /login

Login

Claude Code can be used with your Claude
subscription or billed based on API usage
through your Console account.

Select login method:

❯ 1. Claude account with subscription
   Pro, Max, Team, or Enterprise

  2. Anthropic Console account
     API usage billing

  3. 3rd-party platform
     Amazon Bedrock, Microsoft Foundry, or Vertex AI
```

4. Select **option 1**: "Claude account with subscription"

Browser opens → sign in at claude.ai → click **Allow**

> ***Done.*** *Your Max plan covers Claude Code. No API key needed.*

> ***Seeing errors?*** *Run* `/login` *again and pick option 1. Run* `claude config` *to verify.*

SESSION 1: HOMEWORK

Your Homework: Write the Blueprint

Before Session 2, you're going to create 4 instruction files. These are the foundation of your entire project. Claude Code will read them and know exactly what to build.

1. CLAUDE.md

The master file Claude reads automatically. Your project's brain.

2. Design.md

How the app looks. Pages, colors, buttons, layouts.

3. Web_Architecture.md

How data flows. Which APIs talk to what. The plumbing.

4. API_Design.md

Every API call in detail. What we send, what we get back.

Why homework? Because the quality of these files directly determines how well Claude Code builds your app. Garbage instructions = garbage output. Clear instructions = clean code on the first try.

SESSION 1: HOMEWORK

CLAUDE.md: The Master File

This is the ONLY file Claude Code reads automatically. Everything else, you have to point it to. Get this one right.

Do:

- Start with what the project IS in 2-3 sentences
- List your tech stack explicitly
- Define your data shapes (Lead, Invoice, Project)
- Describe your folder structure
- List commands to run the app
- State rules: "never commit .env"
- Reference your other docs: "See Design.md for layouts"
- Keep it under 2 pages

Don't:

- Write a novel. Claude skims long files.
- Be vague: "make it nice" means nothing
- Skip the data shapes. Claude needs to know what a Lead looks like.
- Forget the rules. Claude will happily commit your API keys if you don't say "never commit .env"
- Duplicate info from other files. Point to them instead.
- Use jargon without explaining it

Design.md: The Visual Blueprint

This file tells Claude what your app looks like. Without it, Claude picks random colors, random layouts, and random components.

Do:

- List every page and its purpose
- Describe the layout: "metric cards on top, table below, chart at bottom"
- Define your color palette with hex codes
- Specify badge colors for each status
- Describe what buttons exist and what they do
- Include a sample Claude Code prompt for each page

Don't:

- Say "make it look modern" with no specifics
- Forget to define status badge colors. Claude will pick its own.
- Skip the navigation structure
- Leave out the empty states ("No leads yet. Sync your Drive.")
- Assume Claude knows what "clean" or "professional" means to you

Pro tip: If you can describe it to a friend over the phone, you can describe it to Claude. "I want 4 boxes at the top showing my key numbers, then a table below with all my leads" is a perfect start.

SESSION 1: HOMEWORK

Web_Architecture.md: The Plumbing

This file maps how data moves through your system. From Google Drive to your dashboard to Gmail.

Do:

- Draw a flow diagram (even in text/ASCII)
- List every API and what it does
- Explain the request flow: "user clicks button, Flask calls service, service calls API"
- Document how authentication works
- List every URL/route in your app
- Explain why you made key decisions ("why Sheets instead of a database?")

Don't:

- Skip the diagram. A picture is worth 1000 words, even an ASCII one.
- Assume Claude knows which layer calls which
- Forget to document the authentication flow
- Leave out error scenarios ("what if the token expires?")
- Mix up "what the app does" with "how the app is built"

SESSION 1: HOMEWORK

API_Design.md: The Recipe Book

This file documents every API call AND how your data objects relate to each other. It's the most detailed file.

Do:

- Show the exact API call with code examples
- Show a sample response (what comes back)
- List common errors with plain-English fixes
- **Draw an object relationship diagram** showing how Lead, Invoice, and Project connect
- **Define the linking field** (company/client name) and explain why it's fuzzy-matched
- **Document the lifecycle**: lead enters > project created > invoice uploaded > lead auto-converts
- Explain design decisions ("why names instead of IDs?")

Don't:

- Just link to external docs. Claude needs the details HERE.
- Skip error handling. Claude will write happy-path-only code.
- **Forget object relationships.** Without them, Claude builds isolated tables that don't talk to each other.
- Leave out auto-update rules (who updates what when a new record arrives)
- Skip the "why" behind design decisions. Claude makes better choices when it understands intent.

Key insight: The object relationships are MORE important than the API calls. Getting Lead > Invoice > Project linking right determines whether your dashboard feels like magic or a disconnected spreadsheet viewer.

Your homework is to draft all 4 files before Session 2. They don't have to be perfect. Start with bullet points. You can refine them with Claude: paste your draft and ask "review this and tell me what's missing."

SESSION 2

Architecture + Data Pipeline

Google APIs, data schemas, and the ETL pipeline

This is where we wire up the data layer

ARCHITECTURE

How It All Connects

```
Google Drive Folders  (/leads, /invoices, /projects)
                              ▼
    OpenAI API  normalizes messy data → clean schema
                              ▼
OpenAI API  cross-references leads ↔ invoices ↔ projects
                              ▼
Google Sheets  master datastore (single source of truth)
                              ▼
        Dashboard  view everything + take action
        ├── Gmail   send emails to leads & clients
            └── Calendar  set invoice reminders
```

Key insight: *Google Drive, Sheets, Gmail, and Calendar share one OAuth credential. One login unlocks all four.*

THE STACK

Five APIs. Two Auth Steps.

API	WHAT IT DOES	AUTH
Google Drive	Scans folders for new/updated spreadsheets	OAuth
Google Sheets	Reads & writes the master datastore	OAuth
Gmail	Sends emails from dashboard (send-only)	OAuth
Calendar	Creates invoice reminders	OAuth
OpenAI	Normalizes data + fuzzy-matches across objects	API Key

Auth Step 1: *One credentials.json from Google Cloud Console unlocks Drive, Sheets, Gmail, and Calendar.*

Auth Step 2: *One API key from platform.openai.com for data normalization.*

BEFORE WE BUILD

What You'll Walk Away With

By the end of this series, you will have:

A Working Dashboard

Running on your computer right now. Open a browser, see your data. Leads, invoices, projects, all in one place.

Files You Own

Every file lives on your machine. No subscription. No vendor lock-in. You can open them, read them, change them.

The Knowledge to Extend It

Want to add Slack notifications? A new data type? You'll know the pattern: describe what you want to Claude Code and iterate.

A GitHub Repo with Everything

All the code, documentation, and sample data. Clone it later, share it with your team, use it as a starting point.

This is not a demo you watch and forget. *You leave with a real thing that works, and the ability to make it do more.*

THE BUILD PLAN

Six Phases. Three Sessions.

Each phase builds on the last. Across three working sessions, we'll connect everything.

1. **Google Cloud Auth Setup**: credentials.json, enable 4 APIs, one OAuth flow — Session 2
2. **Drive Folders + Master Sheet**: Create intake folders and the master spreadsheet — Session 2
3. **ETL Pipeline**: Drive scan → OpenAI normalization → cross-reference → Sheets — Session 2
4. **Dashboard Pages**: Overview, leads, invoices, projects with live data — Session 3
5. **Gmail Integration**: Email a lead or send an invoice from the dashboard — Session 3
6. **Calendar Integration**: Set payment reminders with one click — Session 3

PHASE 1

Google Cloud Auth Setup

Auth is the step that trips everyone up. We're doing it once, live, together.

1. **Google Cloud Console**: console.cloud.google.com
2. **Enable 4 APIs:** Drive, Sheets, Gmail, Calendar
3. **Create Credentials** → OAuth client ID → **Desktop app**
4. **Download JSON immediately**. You will NOT see the secret again
5. **Rename to** `credentials.json` → drop in project folder

High-drama moment: *"You will NOT be able to see the client secret again after closing this window. Download the JSON right now."*

One credentials.json *= Drive + Sheets + Gmail + Calendar. This is the master unlock for the entire Google ecosystem.*

>_ View prompts for this phase

PHASE 1 (CONTINUED)

OpenAI API Key + Environment

Get Your OpenAI Key

1. Go to **platform.openai.com**
2. API Keys → **Create new secret key**
3. Copy it. Shown only once

> *We use **gpt-4o-mini**. A full pipeline run costs less than one cent.*

The .env File

```
# Google
MASTER_SHEET_ID=your_sheet_id
DRIVE_LEADS_FOLDER=folder_id
DRIVE_INVOICES_FOLDER=folder_id
DRIVE_PROJECTS_FOLDER=folder_id

# OpenAI
OPENAI_API_KEY=sk-your-key

# Flask
FLASK_SECRET_KEY=random-string
```

This file holds all your secrets. Never commit it. Never share it.

> ***Where do I find the Sheet ID?*** *Open your Google Sheet. Look at the URL:* `docs.google.com/spreadsheets/d/this-long-string/edit`*. The long string between* `/d/` *and* `/edit` *is your Sheet ID. Same pattern for Drive folder IDs in the URL.*

PHASE 2

Drive Folders + Master Sheet

Set up the intake system and the master datastore.

Google Drive Structure

```
BusinessHub/
  ├── leads/        ← drop lead spreadsheets
  ├── invoices/     ← drop invoice spreadsheets
  └── projects/     ← drop project spreadsheets
```

Create or upload spreadsheets from any device. Drive syncs them automatically.

Master Google Sheet

One spreadsheet. Three tabs:

Leads name, email, company, source, stage, created_at

Invoices invoice_number, client, amount, due_date, status

Projects name, client, budget, total_billed, deadline, status

Why Sheets? Everyone knows how to open a spreadsheet. If something breaks, you can see exactly where.

>_ View prompts for this phase

PHASE 2 (CONTINUED)

The Data Objects

Three object types. Consistent schemas. Connected by company name.

Lead

name	Jane Smith
email	jane@acme.com
company	Acme Corp
source	Referral
stage	new

Invoice

invoice_number	INV-0042
client	Acme Corp
amount	$4,500
due_date	2026-04-15
status	sent

Project

name	Website Redesign
client	Acme Corp
budget	$15,000
total_billed	$4,500
status	active

The connection: *All three objects link by* ***company/client name****. When an invoice arrives for "Acme Corp", the pipeline finds the matching lead and project automatically.*

PHASE 3

The ETL Pipeline

Extract. Transform. Load. This is how raw spreadsheets become structured, linked data.

- **E** **Extract**: Drive API scans folders for new or updated spreadsheets
- **T** **Transform**: OpenAI normalizes messy columns into canonical schemas, then cross-references across all three object types
- **L** **Load**: Write clean, linked data to the master Google Sheet

> ***Key teaching moment:*** *"What we are building is called an ETL pipeline. Real data engineers build these every day. By the end of this series, so will you."*

>_ View prompts for this phase

PHASE 3 (CONTINUED)

OpenAI Normalization

Messy spreadsheet in. Clean, structured data out.

Before (raw spreadsheet)

```
Headers:
  "Full Name", "Email Address",
  "Business Name", "How Found",
  "Date Added"

Rows:
  "Jane Smith", "jane@acme.com",
  "Acme Corporation", "LinkedIn",
  "April 1, 2026"
```

Wrong column names. Inconsistent date format. Missing fields.

After (normalized)

```
{
  "name": "Jane Smith",
  "email": "jane@acme.com",
  "company": "Acme Corporation",
  "source": "LinkedIn",
  "stage": "new",
  "created_at": "2026-04-01"
}
```

Canonical schema. Clean date. Default stage applied. Ready for Sheets.

Why OpenAI? *"Full Name" → `name`. "Business Name" → `company`. "April 1, 2026" → "2026-04-01". A language model handles this naturally. Regex would break on the first edge case.*

PHASE 3 (CONTINUED)

Cross-Reference Reconciliation

This is where your data starts talking to itself.

```
New invoice arrives: { client: "Acme Corporation", amount: $4,500 }
▼ search Leads tab → Found: "Acme Corp" (stage: "contacted") → update stage → "converted"
▼ search Projects tab → Found: "Website Redesign" (client: "Acme Corp") → total_billed += $4,500
```

> ***Fuzzy matching:*** *"Acme Corporation" and "Acme Corp" are obviously the same company to you. And to OpenAI. But* `==` *says they're different. This is why we use a language model.*

TRIGGER	LOOKUP	AUTO-UPDATE
New invoice	Match lead by client	Lead stage → "converted"
New invoice	Match project by client	Project total_billed += amount
New lead	Match project by company	Link lead to project
New project	Match leads + invoices	Associate existing records

SESSION 3

Dashboard + Actions

Building pages, sending emails, setting reminders

This is where it all comes together

PHASE 4

Building the Dashboard

Four pages. Real data. Built by describing what you want in plain English.

/dashboard

Overview: 4 metric cards (leads, revenue, invoices, projects), revenue chart, recent activity feed, "Sync Drive" button

/leads

Filterable lead table with stage badges (new, contacted, converted). "Email" button per row.

/invoices

Invoice table with status badges. "Send Invoice" and "Set Reminder" action buttons.

/projects

Card layout with budget progress bars. Linked lead and invoice counts. Click to expand.

>_ View prompts for this phase, one per page

HOW WE BUILD

The Golden Formula

Every prompt to Claude Code follows one pattern:

Context

Where you are and what's happening.

"I'm on the /invoices page"

Goal

The outcome you want.

"Add a Set Reminder button on overdue invoices"

Constraints

What NOT to change.

"Don't change the table layout"

Full example:

> *"I'm on the /invoices page. Overdue invoices need a 'Set Reminder' button that creates a Google Calendar event for the due date. Use the existing calendar service and keep the table layout as-is."*

> ***When Claude gets it wrong**, this is the best teaching moment. Say: "Not quite, use due_date, not created_at." The audience sees that iteration is normal, not failure.*

PHASE 5

Gmail Integration

Email a lead or send an invoice, without leaving the dashboard.

How It Works

1. User clicks **"Email"** on a lead row
2. Compose modal opens, **pre-filled** with lead's email
3. User types subject + body, clicks **Send**
4. Flask calls Gmail API → email sent from your account

> ***Send-only scope.*** *The app can send emails but cannot read your inbox. Safe by design.*

>_ View prompts for Gmail

Under the hood:

```
POST /api/email/send
{
  "to": "jane@acme.com",
  "subject": "Following up",
  "body": "Hi Jane, ..."
}

→ Gmail API: messages.send()

→ { "status": "sent" }
```

PHASE 6

Calendar Integration

One click = invoice due date on your Google Calendar with a 24-hour reminder.

How It Works

1. User clicks **"Set Reminder"** on an invoice
2. Flask creates an all-day Calendar event on the due date
3. Popup + email reminder fires **24 hours before**
4. Returns a direct link to the Calendar event

> ***The moment it clicks:*** *The audience sees that the dashboard isn't just displaying data. It's doing things. Real actions in real tools they use every day.*

What Gets Created

Calendar Event

Title	Invoice Due: INV-0042 - Acme Corp ($4,500)
Date	April 15, 2026
Reminder	24h before (popup + email)

>_ **View prompts for Calendar**

THE MOST IMPORTANT SLIDE

When It Breaks

SAY THIS: "Watch what I do here. This is the most important part of the demo. I am not fixing code. I am describing the problem in plain English and letting Claude Code figure it out."

Example debugging prompts:

PROBLEM	WHAT YOU SAY TO CLAUDE CODE
App won't start	"app.py crashes on startup. Read the error and fix it."
Page shows no data	"/leads is empty but the Sheet has data. Debug this."
Email fails	"Send email returns 403. Gmail API is enabled. Check scopes."
Cross-ref missed	"Invoice for Acme didn't update the lead. Check reconciliation."
OpenAI returns junk	"Normalization put company name in the email field. Fix the prompt."

Narrating through a real error is the most valuable thing a non-technical audience can watch. *It shows that errors are normal, and that describing the problem clearly is a real, transferable skill.*

UNDER THE HOOD

Clean Architecture

This is how the app stays organized, so when something breaks, you know exactly where to look.

Every request flows through four layers. Each layer has one job.

Route

"What URL was hit?" Maps URLs to controllers. Thin wiring, nothing else.

Controller

"What did the user send?" Reads the request, calls the right service, returns a response.

Service

"What should happen?" Business logic, API calls, data processing. The brains.

Repository

"Where is the data?" Reads and writes to files or Google Sheets. Data access only.

Route → Controller → Service → Repository

> ***Why this matters:*** *When something breaks, you know exactly which layer to look at. The route didn't match? The controller read the wrong field? The service called the wrong API? The repository wrote to the wrong file? One layer, one problem.*

CLOSING

Three Takeaways

1. **You don't need to understand the code. You need to understand what you want.**
   Claude Code writes the code. You direct it. The skill is knowing what your data should look like and what you want to do with it.

2. **Define your objects. Define your relationships. The rest follows.**
   The real work isn't coding. It's deciding what a "lead" is, what an "invoice" is, and how they connect. Once that's clear, the pipeline builds itself.

3. **A messy first attempt you can iterate on beats waiting for a perfect plan.**
   Ship something. Click around. See what's wrong. Describe the problem to Claude Code. Iterate. That's the process.

Leave them with this:

"You don't need to understand how it works. You just need to know what you want, and where your work lives."

What data source would *you* connect if this was your project?

Resources shared after session • All code on GitHub • Documentation included

---

Session 4 (Optional): Cleanup, polish, and Q&A. Bring your bugs, your questions, and your ideas.